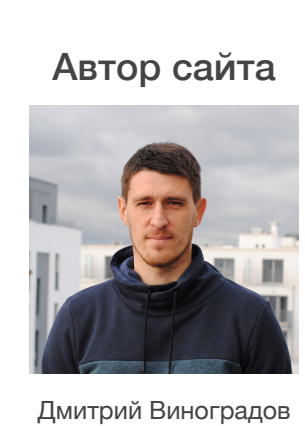
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50%
на набор курсов:
Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 31. Анимация. AnimationSpec

18 сентября 2023

В этом уроке рассматриваем доступные нам спецификации AnimationSpec: tween, spring, keyframes, repeatbale, snap

На прошлых уроках мы подробно познакомились с иерархией механизмов анимации:

Animatable -> TargetBasedAnimation -> AnimationSpec (спецификация)

Animatable запускает цикл, в котором (каждый кадр) использует TargetBasedAnimation, чтобы конвертировать время в значение. (например, 500 мсек - в 100 dp).

Но TargetBasedAnimation не сам выполняет эти вычисления. Если помните, при создании TargetBasedAnimation мы передаем ему спецификацию:

```
TargetBasedAnimation(
   animationSpec = tween(1000, easing = LinearEasing),
   typeConverter = ...,
   initialValue = …,
   targetValue = …
)
```

TargetBasedAnimation занимается конвертацией DP в вектор и обратно. А все вычисления выполняет именно спецификация.

Давайте подробно рассмотрим, какие спецификации нам доступны. В качестве примера будем использовать примерно тот же экран, что и на прошлом уроке:

```
@Composable
fun HomeScreen() {
   val myAnimation = remember { Animatable(
       initialValue = 0.dp,
       typeConverter = Dp.VectorConverter
   )}

   Spacer(modifier = Modifier
       .background(Color.Green)
       .height(50.dp)
       .width(myAnimation.value)
   )

   LaunchedEffect(Unit) {
       delay(1000)
       myAnimation.animateTo(
           targetValue = 250.dp,
           animationSpec = createSpec()
       )
   }
}

private fun createSpec(): AnimationSpec<Dp> =
   tween(
       durationMillis = 1000,
       easing = LinearEasing
   )
```

Создаем Animatable и используем его для анимации ширины элемента Spacer с зеленым фоном.

Анимацию стартуем в LaunchedEffect. Паузу в 1000 мсек я добавил, чтобы приложение гарантировано отобразилось на экране и мы успели увидеть начало анимации.

Создание спецификации вынесено в отдельный метод createSpec. Именно здесь мы и будем экспериментировать с разными видами спецификаций.

tween

Эту спецификацию мы уже использовали в прошлых примерах:

```
tween(
   durationMillis = 1000,
   easing = LinearEasing
)
```

От нас тут требуется указать, как долго должна длиться анимация и по какой функции (Easing) она будет рассчитываться. Мы обычно выбирали LinearEasing - это постоянная скорость.

Результат:

Давайте рассмотрим подробнее, что это за Easing такой и какие у него еще есть варианты.

Easing

Интерфейс Easing выглядит так:

```
fun interface Easing {
   fun transform(fraction: Float): Float
}
```

Метод transform принимает на вход время, а вернуть должен значение. Т.е. именно этот метод и расчитывает значение анимации.

Откроем исходники LinearEasing:

```
val LinearEasing: Easing = Easing { fraction -> fraction }
```

Это очень простая реализация интерфейса Easing, LinearEasing просто возвращает то же самое значение, что получает. Звучит странно, с учетом того, что получает он мсек, а вернуть должен, например, DP.

Но тут есть важный нюанс: fraction это дробное число от 0 до 1. И вернуть необходимо тоже дробное число от 0 до 1. Чтобы лучше это понять давайте рассмотрим конкретный пример.

Пусть у нас есть tween анимация с LinearEasing, которая на протяжении **5000 мсек** анимирует значение от **0 dp до 100 dp**.

Для примера возьмем несколько моментов времени и посмотрим, как для них рассчитывается анимация.

Т.е. в момент времени 1000 мсек должно быть рассчитано значение 20 dp
2000 мсек - 40 dp
5000 мсек - 100 dp

Давайте смотреть, как именно они рассчитываются.

Наступает 1000-я миллисекунда анимации, и Easing должен рассчитать значение анимации на этот момент времени. На вход Easing передается время, но это будет не 1000, а 0.2. Откуда это берется?

tween берет общую длительность анимации 5000 мсек и считает его единицей. Тогда 1000 мсек по этой шкале превращается в 0.2. Это число и передается в Easing как время.

LinearEasing недолго думая просто возвращает это же число обратно. Т.е. tween передал время 0.2 и обратно получает число 0.2, которое надо как-то превратить в значение анимации (DP). Используется тот же принцип, что и со временем, только в обратную сторону. Диапазон от 0 до 1 конвертируется в диапазон значений анимации от начального (0 dp) до конечного (100 dp). И 0.2 превращается в 20.

Таким образом анимация для времени 1000 получила значение 20 dp.

Аналогично рассчитываются и другие моменты времени.

Время 2000 мсек конвертируется в 0.4 и передается в LinearEasing. Полученное в ответ число 0.4 конвертируется в 40.

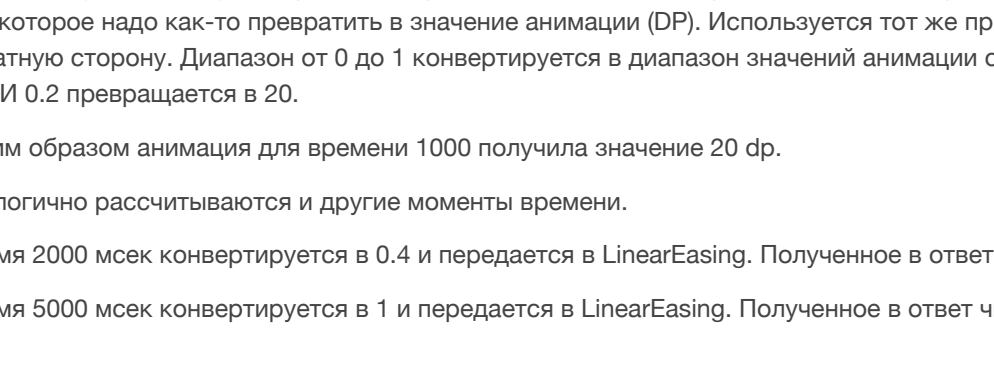
Время 5000 мсек конвертируется в 1 и передается в LinearEasing. Полученное в ответ число 1 конвертируется в 100.

Т.е. получается такая иерархия:

TargetBasedAnimation -> tween -> Easing

На пути от TargetBasedAnimation -> tween выполняется конвертация DP в векторы и обратно. А на пути tween -> Easing векторы приводятся к значениям 0..1 и обратно.

Кроме LinearEasing нам доступны еще несколько стандартных Easing. Они работают по принципу кривых Безье.



```
Elements that begin and end at rest use this standard easing. They speed up quickly and slow down
gradually, in order to emphasize the end of the transition.
Standard easing puts subtle attention at the end of an animation, by giving more time to deceleration
than acceleration. It is the most common form of easing.
This is equivalent to the Android FastOutSlowInInterpolator
val FastOutSlowInEasing: Easing = CubicBezierEasing( a: 0.4f,  b: 0.0f,  c: 0.2f,  d: 1.0f)

Incoming elements are animated using deceleration easing, which starts a transition at peak velocity
(the fastest point of an element's movement) and ends at rest.
This is equivalent to the Android LinearOutSlowInInterpolator
val LinearOutSlowInEasing: Easing = CubicBezierEasing( a: 0.0f,  b: 0.0f,  c: 0.2f,  d: 1.0f)

Elements exiting a screen use acceleration easing, where they start at rest and end at peak velocity.
This is equivalent to the Android FastOutLinearInInterpolator
val FastOutLinearInEasing: Easing = CubicBezierEasing( a: 0.4f,  b: 0.0f,  c: 1.0f,  d: 1.0f)
```

В результате скорость анимации будет не постоянной. Она может быть быстрой в начале и медленной ближе к концу, или наоборот.

С различными вариантами такой анимации можно поэкспериментировать онлайн: https://cubic-bezier.com/#0,.5,1,.5

При желании, мы можем создать свою реализацию Easing.

Вот, например, Easing, при использовании которого, мы получим не пллавную анимацию, а по шагам:

```
val MyEasing: Easing = Easing { fraction ->
   when {
       fraction < 0.25 -> 0F
       fraction < 0.5 -> 0.25F
       fraction < 0.75 -> 0.5F
       fraction < 1 -> 0.75F
       else -> 1F
   }
}
```

Используем ее в спецификации в нашем методе createSpec

```
tween(
   durationMillis = 1000,
   easing = MyEasing
)
```

Результат:

Easing выдает новое значение для анимации только по достижению определенных значений времени.

На этом, пожалуй, все про спецификацию tween. Осталось только добавить, что можно задать паузу перед стартом с помощью параметра delayMillis:

```
tween(
   delayMillis = 500,
   durationMillis = 1000,
   easing = LinearEasing
)
```

spring

Это спецификация рассчитывает анимацию так, чтобы визуально это выглядело как пружина.

Пример:

```
spring(
   dampingRatio = Spring.DampingRatioLowBouncy,
   stiffness = Spring.StiffnessLow
)
```

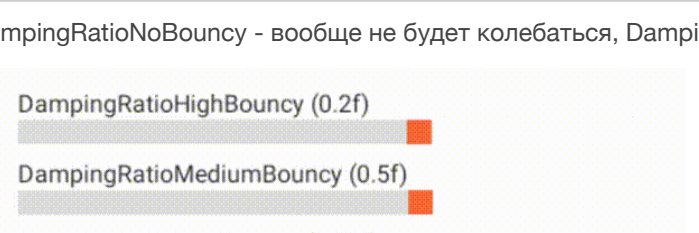
Результат:

Параметр dampingRatio - это затухание, т.е. насколько сильно значение будет колебаться туда-сюда в конце анимации.

Есть 4 константы:

```
const val DampingRatioHighBouncy = 0.2f
const val DampingRatioMediumBouncy = 0.5f
const val DampingRatioLowBouncy = 0.75f
const val DampingRatioNoBouncy = 1f
```

DampingRatioNoBouncy - вообще не будет колебаться, DampingRatioHighBouncy - это очень высокая степень колебаний


DampingRatioHighBouncy (0.2f)
DampingRatioMediumBouncy (0.5f)
DampingRatioLowBouncy (0.75f)
DampingRatioNoBouncy (1f)
(dampingRatio = 2f)

Сейчас в примере задана низкая степень колебаний. Поставим высокую:

```
spring(
   dampingRatio = Spring.DampingRatioHighBouncy,
   stiffness = Spring.StiffnessLow
)
```

Результат:

Параметр stiffness - это жесткость пружины, т.е. насколько быстро она будет разжиматься.

Для этого параметра также есть несколько констант:

```
const val StiffnessHigh = 10_000f
const val StiffnessMedium = 1500f
const val StiffnessMediumLow = 400f
const val StiffnessLow = 200f
const val StiffnessVeryLow = 50f
```

StiffnessHigh - очень быстро, а StiffnessVeryLow - очень медленно

В нашем прошлом примере жесткость была низкая, соответственно скорость - не очень быстрая.

Поставим высокую жесткость:

```
spring(
   dampingRatio = Spring.DampingRatioHighBouncy,
   stiffness = Spring.StiffnessHigh
)
```

Это увеличит скорость. Результат:

Есть еще параметр visibilityThreshold. Это минимально разрешенный размер колебаний. Как только колебания становятся меньше этого значения, то анимация этих колебаний прекращается.

Т.е. если поставить 1.dp, то будет колебаться долго, пока колебания не затухнут до совсем минимальных значений в 1.dp

```
spring(
   dampingRatio = Spring.DampingRatioHighBouncy,
   stiffness = Spring.StiffnessVeryLow,
   visibilityThreshold = 1.dp
)
```

Результат:

А если поставить 30.dp, то будет колебаться недолго, т.к. диапазон колебаний достаточно быстро станет меньше, чем 30:

```
spring(
   dampingRatio = Spring.DampingRatioHighBouncy,
   stiffness = Spring.StiffnessVeryLow,
   visibilityThreshold = 30.dp
)
```

Результат:

Заметьте, что у спецификации spring нет возможности задать duration. Бывает и так. Мы можем только повлиять на скорость с помощью stiffness, но не указать точную продолжительность.

keyframes

Эта спецификация позволяет задавать не только продолжительность анимации, но и контрольные точки. Т.е. в какое время какое значение анимации должно быть.

Пример:

```
keyframes {
   durationMillis = 1000
   200.dp at 300
}
```

Анимация длится 1000 мсек и должна достигнуть 200 dp в момент времени 300 мсек

Результат:

Видно, что первые 200 dp проходят быстро (за 300 мсек), а остальные 50 dp идут оставшиеся 700 мсек.

Точек может быть несколько. А вместо конкретного значения времени, можно использовать коэфициенты:

```
keyframes {
   durationMillis = 5000
   50.dp atFraction 0.4f
   200.dp atFraction 0.6f
}
```

0.4f от 5000 мсек - это 2000 мсек, а 0.6f - это 3000 мсек.

Под капотом у этой спецификации также используется Easing. По умолчанию - это LinearEasing, т.е. постоянная скорость.

Мы можем указать отдельный Easing для каждого шага анимации с помощью with:

```
keyframes {
   durationMillis = 5000
   50.dp at 2000 with MyEasing
   200.dp at 3000
}
```

Используем MyEasing из примера про tween. Обратите внимание, что Easing применится к шагу, который будет после 2000 мсек, а не до.

Результат:

от 0 мсек до 2000 мсек использовался LinearEasing по умолчанию
от 2000 до 3000 - наш MyEasing
от 3000 до конца - снова LinearEasing по умолчанию

Если надо применить Easing к самому первому шагу, то просто добавляем в начало точку с нулевым временем.

```
keyframes {
   durationMillis = 5000
   0.dp at 0 with MyEasing
   50.dp at 2000
   200.dp at 3000
}
```

MyEasing будет применен к шагу от 0 мсек до 2000 мсек.

Точки можно ставить и в обратном направлении. Например, дойдем до 200 dp, потом обратно до 150 dp, а затем уже до конца

```
keyframes {
   durationMillis = 5000
   200.dp at 3000
   150.dp at 4000
}
```

Результат:

При необходимости можно задавать точки программно.

Пример анимации которая идет вперед, и иногда делает шаг назад:

```
keyframes {
   durationMillis = 5000
   for (i in 1..5) {
       (20.dp + 50.dp * i) at (1000 * i - 200)
       (50.dp * i) at 1000 * i
   }
}
```

Результат:

У спецификации keyframes также есть параметр delayMillis для задержки.

repeatable

Эта спецификация позволяет организовать повторное выполнение для других спецификаций.

Пример:

```
animationSpec = repeatable(
   iterations = 3,
   animation = tween(durationMillis = 1000),
   repeatMode = RepeatMode.Reverse
)
```

Спецификация tween будет выполнена три раза: туда, обратно и снова туда

Результат:

Если сменить режим повтора с Reverse на Restart, то получим анимацию три раза в одну сторону:

Результат:

У этой спецификации также есть параметр для задания задержки: initialStartOffset. Причем эту задержку можно сделать и отрицательной. Тогда анимация начнется не с начала, а с конкретной временной позиции.

Если нужна бесконечная анимация, то используем infiniteRepeatable

```
animationSpec = infiniteRepeatable(
   animation = tween(durationMillis = 1000),
   repeatMode = RepeatMode.Reverse
)
```

snap

Эта спецификация просто сразу ставит конечное значение.

```
snap()
```

Результат

Не очень понятно зачем нужна такая "анимация", но видимо есть какие-то кейсы.

Из параметров у нее только задержка - delayMillis

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад Вперёд

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте формы обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.

Font Awesome font licensed under SIL OFL 1.1.